



SERVIÇO PÚBLICO FEDERAL · MINISTÉRIO DA EDUCAÇÃO
UNIVERSIDADE FEDERAL DE VIÇOSA · UFV
CAMPUS FLORESTAL

Trabalho Prático - Parte 4: Caldeirão

Autores (as):

Sabrina Bruni de Souza Faria [EF 05081]

Luiz Guilherme de Sá Gontijo [EF 04839]

Breno Júnio de Oliveira Gomes [EF 05085]

Disciplina: Sistemas Distribuídos e Paralelos [CCF - 355]

Sumário

1. Introdução 🧑	3
2. Melhorias 📈	3
3. Middleware ✨	3
3.1. Cliente:	4
3.2. Servidor de Aplicação:	4
3.3. Servidor de Banco de dados:	5
4. Resultados 🏊	7
5. Vantagens 🧰	9
6. Dificuldades 🧑	9
7. Conclusão ⬅️	9
8. Referências 📖	9

1. Introdução 🧙‍♂️:

Esse trabalho tem como objetivo apresentar a implementação em Python de um sistema distribuído de Marketplace de Poções Mágicas. Esse projeto foi adaptado de troca de mensagens utilizando API de sockets para uso de Middleware Pyro5. Com isso, essa documentação busca explicar as alterações, as facilidades e dificuldades ao realizar essa adaptação.

2. Melhorias 📈:

Em relação a entrega anterior, com API de sockets, esse projeto foi melhorado com funcionalidades que não haviam sido implementadas ou completadas anteriormente devido à falta de tempo e complexidade ao criar o projeto do zero. Agora, com somente a necessidade de adaptação, foi possível adicionar todas as funcionalidades que haviam sido planejadas inicialmente.

Sendo assim, o Marketplace se encontra com as funcionalidades de pedido completas, podendo realizar a visualização, confirmação e cancelamento dos pedidos sem problemas, e todas as funcionalidades de exclusão estão funcionando corretamente.

Além disso, adicionamos a funcionalidade de logout e excluir perfil, separamos os pedidos do usuário entre confirmados e em andamento, adicionamos algumas mensagens na interface para melhorar a experiência do usuário e melhoramos as mensagens enviadas por email.

3. Middleware ✨:

Com o uso do middleware, tivemos que mudar o fluxo de troca de mensagens para retorno de função. Dessa forma, tivemos que mudar alguns padrões para que fizessem mais sentido usando o retorno. Por exemplo, ao solicitar a visualização de um item com imagem, os bytes dessa imagem são solicitados e retornados em uma requisição diferente. Porém, ao criar ou editar um item com imagem, a imagem vem junto ao objeto em uma tupla.

Essa decisão foi tomada assim pois, ao criar um objeto, há imagens específicas associadas a ele, mas ao visualizar objetos, onde muitas vezes vêm vários objetos numa lista, o retorno dessas imagens acaba ficando trabalhoso e confuso, sendo mais fácil recuperar as imagens separadamente.

Entretanto, mantivemos a ideia de enviar os objetos na forma de dicionário, porém sem a conversão para json. Isso porque o middleware já converte para dicionário ao enviar, então na chegada teríamos que converter de dicionário para objeto de todo modo. Portanto, decidimos manter a conversão manual para dicionário, a fim de termos maior controle dos parâmetros que estão sendo de fato enviados.

3.1. Cliente:

Na parte do processo cliente, foi necessário substituir a parte do socket pelo middleware, substituindo as funções de criação, conexão e desconexão do socket por funções de encontrar o name server e de conexão e desconexão do Proxy.

Além disso, a parte de aplicação do cliente teve que ser substituída para que utilizasse as funções referentes ao middleware ao invés do socket.

Na versão de socket, se o servidor não respondesse a alguma requisição devido a timeout, o usuário poderia continuar usando o sistema e pedir outras coisas, mas se a conexão falhasse ao criar um socket, ele não conseguiria fazer as requisições mesmo com a interface abrindo normalmente. Assim, como as threads do middleware não tratam as questões de interface, foi preciso manter as threads do cliente. Com isso, nessa nova configuração, o Proxy é criado a cada nova requisição, visto que um mesmo proxy não pode ser utilizado em threads diferentes.

Dessa forma, foi feito um tratamento para que, ao ser aberto, o programa encontre a uri e conecte e desconecte a cada requisição. Se o servidor de aplicação for reiniciado enquanto o programa está aberto, a uri encontrada anteriormente se torna inválida, e então o programa encontra ela novamente antes de criar o proxy para a requisição. Isso foi importante para manter o programa funcionando normalmente sem que o usuário tenha que fechar e abrir o programa de novo por qualquer motivo que seja.

3.2. Servidor de Aplicação:

Duas partes do servidor de aplicação sumiram do sistema na transição para o middleware: O Client Handler e o Callback. O Client Handler era a parte responsável por lidar com as mensagens enviadas do cliente para o servidor. Como o sistema não é mais baseado em troca de mensagens, e sim em chamadas de funções remotas, o papel do Client Handler se perdeu. Agora, para visualizar melhor o funcionamento com invocações remotas, o cliente recebe sua resposta como retorno da função chamada pelo servidor, ou seja, não é mais necessária a parte de Callback que era antes responsável por preparar a resposta do Servidor para o Cliente.

Agora, o sistema fica dividido em duas partes apenas: a Fachada de Serviços do Servidor e a Fila de Mensagens. A Fachada cumpre o papel de disponibilizar as funções do Servidor expostas via Pyro. A Fila de Mensagens continua cumprindo seu papel anterior: enfileirar e afunilar as requisições do Cliente para Banco de Dados. A diferença é que, agora, ela não somente guarda as requisições do Cliente para enviar para o Banco de Dados como também guarda a resposta do Banco para retornar pro Cliente.

A Fachada de Serviços do Servidor, representada pela classe `ServiçosServidorAplicação`, é uma *Facade* de todas as funções que um Cliente pode chamar do Servidor, tendo a mesma aparência do decisor do extinto Client Handler.

Com base na requisição do Cliente, a devida função será chamada da classe da operação relacionada.

Outra diferença é que, agora, cada operação não abre uma nova thread explicitamente com a biblioteca Threading.thread. Tudo o que é relacionado à threads e envolve a comunicação entre Cliente e Servidor ficou nas mãos da implementação interna do Pyro. No caso, Pyro já abre threads diferentes para cada chamada remota.

A Fila de Mensagens ainda cumpre seu papel de enfileirar as requisições feitas pelo Cliente. Ela ainda roda paralelamente ao sistema em outra thread. O que mudou é que, agora, a estrutura Fila de Mensagens também guarda a resposta do Banco de Dados para o Servidor pegar depois e enviar para o Cliente que fez aquela requisição.

O funcionamento geral do Servidor agora é o seguinte:

- A função de uma operação é chamada;
- O Servidor prepara essa requisição, cria um identificador para essa requisição, registra ela e a enfileira. Depois de enfileirar a requisição, o Servidor fica esperando a resposta para esta requisição chegar do Banco;
- Quando o Banco não estiver ocupado e não houver nenhuma outra requisição na frente dela na fila, a Fila de Mensagens envia a requisição do Cliente para o Banco de Dados;
- Quando o Banco responder, a Fila guarda a resposta e avisa o servidor que uma resposta chegou do Banco de Dados;
- A função que o Cliente chamou, que estava em espera, vê que sua resposta chegou, recupera-a e a retorna-a ao Cliente.

Enfim, o uso do middleware simplificou todo o processo de comunicação do Servidor com os outros processos. Apesar de a descrição fazer parecer que o sistema mudou muito, só a parte de comunicação foi trocada de vários passos de tratamento de mensagem para simples chamadas de funções. O código do Servidor já estava modelado para receber uma refatoração na sua estrutura de comunicação sem se fazer necessária uma completa reformulação do Servidor de Aplicação.

3.3. Servidor de Banco de dados:

O servidor de banco de dados foi a parte que mais passou por refatorações nesta parte do trabalho. Isso já era esperado pois ainda era necessário implementar as funcionalidades que faltaram na última entrega, porém, devido a uma engenharia incipiente na fase de planejamento do banco, o desenvolvimento das funcionalidades que faltavam esbarrou em inúmeras inviabilidades e contradições que não haviam sido previstas corretamente, tudo junto com a necessidade de reestruturar totalmente o modo como o servidor do banco interage com o servidor de aplicação na implementação com middleware.

Primeiramente, o servidor de banco de dados estava estruturado como uma fachada, usando um switch-case para tratar as mensagens de requisição recebidas, tratando ele mesmo o passo a passo lógico da resposta e fazendo a interação via outra classe de fachada, o Banqueiro. Para passar a usar o middleware, era preciso transformar cada switch-case numa função, e como o código com switch-case estava muito acoplado, foi decidido que a classe Banqueiro seria a fachada a conversar com o servidor de aplicação, fazendo do servidor apenas o código que exporia o Banqueiro via Pyro5.

Isso fez com que a classe Banqueiro aumentasse muito em tamanho. Além disso, foi percebida a necessidade de modularizar melhor o modo como o código lidava com imagens, levando a criação do arquivo `image_utils`. Outra melhoria notável é o uso de logging, que passou de inexistente para uso moderado, mas, por questões de tempo, só foi possível padronizar isso dentro dos métodos do Banqueiro.

A maior dificuldade, no entanto, foi, após essa transição, implementar as funcionalidades relativas à entidade Pedido. A regra de negócio é que um pedido não pode desaparecer após ser confirmado. Porém, como as informações totais de um pedido ainda dependem do anúncio, do produto e do endereço do comprador, só é possível apagar, por exemplo, um produto, caso não houvesse pedidos confirmados que dependessem de algum anúncio desse produto. Essa forte dependência levou a várias tentativas de reestruturação dessa entidade, até ser decidido que a melhor solução seria criar entidades de redundância, separando os pedidos confirmados em uma entidade distinta da dos pedidos em andamento.

A solução foi, basicamente, que, se um pedido for confirmado, as informações de seu anúncio, produto e endereço serão copiadas em tabelas de redundância, passando a ser independentes das informações que os originaram. Isso não apenas resolve inúmeros impedimentos na hora de excluir elementos dependentes de pedidos confirmados no banco, como também faz mais sentido, pois um pedido confirmado não poderia, por exemplo, ter seu endereço alterado – Mas sem as tabelas de redundância, ele estava relacionado ao mesmo endereço que o usuário podia visualizar e editar.

O acesso ao banco, no começo, buscava ser um código inteligente, sendo baseado em subrotinas que montavam os comandos SQL feitos ao banco conforme parâmetros e que classes filhas estavam usando-as. Entretanto, porém isso acabou limitando a complexidade dos comandos SQL, impedindo a criação de comandos muito específicos, que passaram a ser necessários na nova versão do banco. Como era tarde demais para refatorar todo o acesso ao banco, os comandos mais complexos, em geral referentes à exclusão, acabaram ficando pouco eficientes, o que se torna um problema conforme a população de dados escala.

4. Resultados 🏠:

O resultado final da transição, felizmente, é um Marketplace de poções totalmente funcional e, em boa parte, escalável. Todas as funcionalidades previstas conseguiram ser implementadas, sendo elas todos os casos de uso do diagrama montado nas entregas anteriores.

Conforme o diagrama de Caso de Uso, atualizado abaixo, o sistema cumpre as tarefas básicas especificadas para o trabalho:

- Os usuários podem criar e manipular um cadastro de si próprios no sistema, podendo realizar login e logout;
- Criar uma loja, na qual podem criar produtos, e deles, criar anúncios, que serão visíveis a outros usuários se despausados;
- Usuários poderão realizar pedidos a partir de anúncios, que podem (ou não) ser efetivados pelo vendedor do anúncio após o recebimento do pagamento (o sistema usa pix, mas a detecção de pagamento não é automática);
- Cada usuário pode visualizar os pedidos que fez e os pedidos feitos para cada uma de suas lojas, que estão, em ambos os casos, divididos entre pedidos em andamento e pedidos confirmados. O vendedor pode tanto cancelar quanto confirmar pedidos em andamento de suas lojas.
- Todos esses elementos podem ser criados, editados e excluídos: Usuários, lojas, produtos e anúncios. Pedidos em andamento podem ser criados ou cancelados, mas não editados. Pedidos confirmados são permanentes – não podem ser excluídos ou editados. Usuários podem ser editados e criados, contanto que não haja e-mail ou CPF iguais para usuários distintos.
- Anúncios são pausados automaticamente quando o estoque de seu produto acaba – O vendedor deve excluir o anúncio ou aumentar sua quantidade de produto disponível.

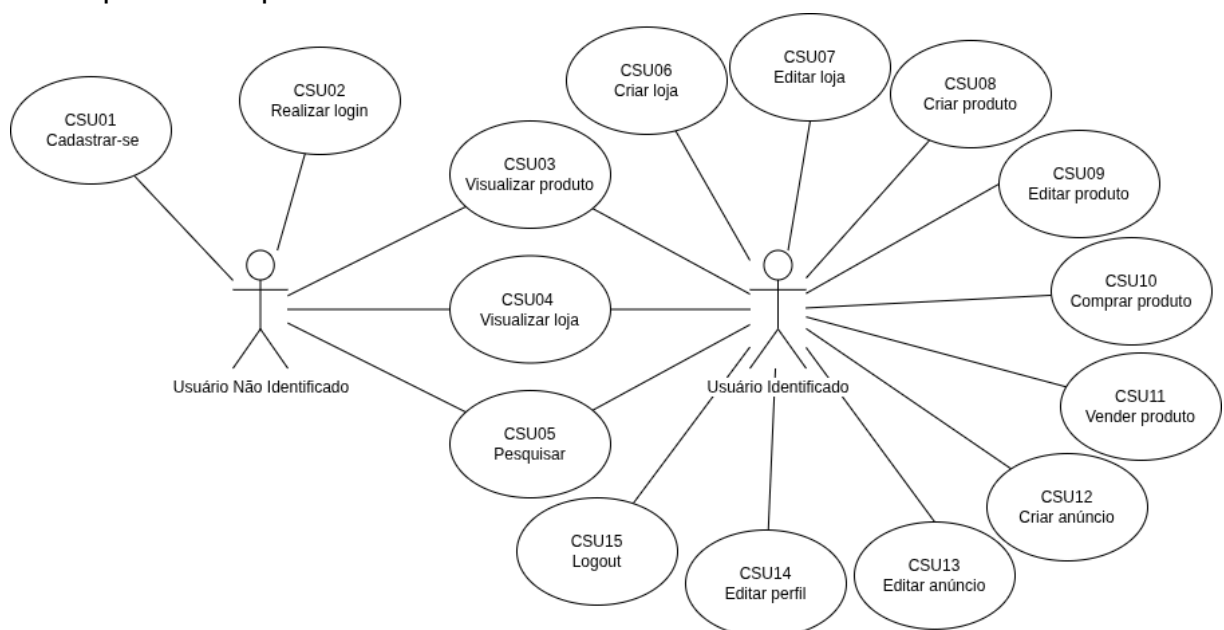


Figura: Diagrama de Casos de Uso

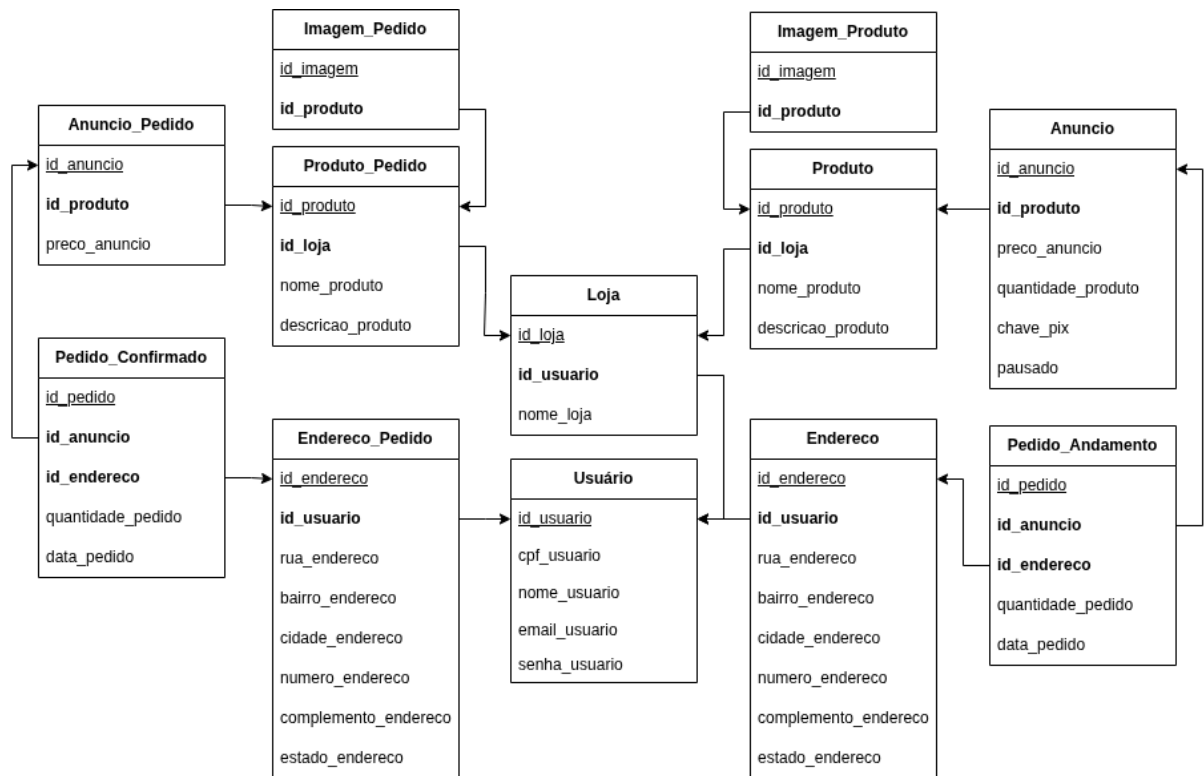


Figura: Diagrama de Banco de Dados

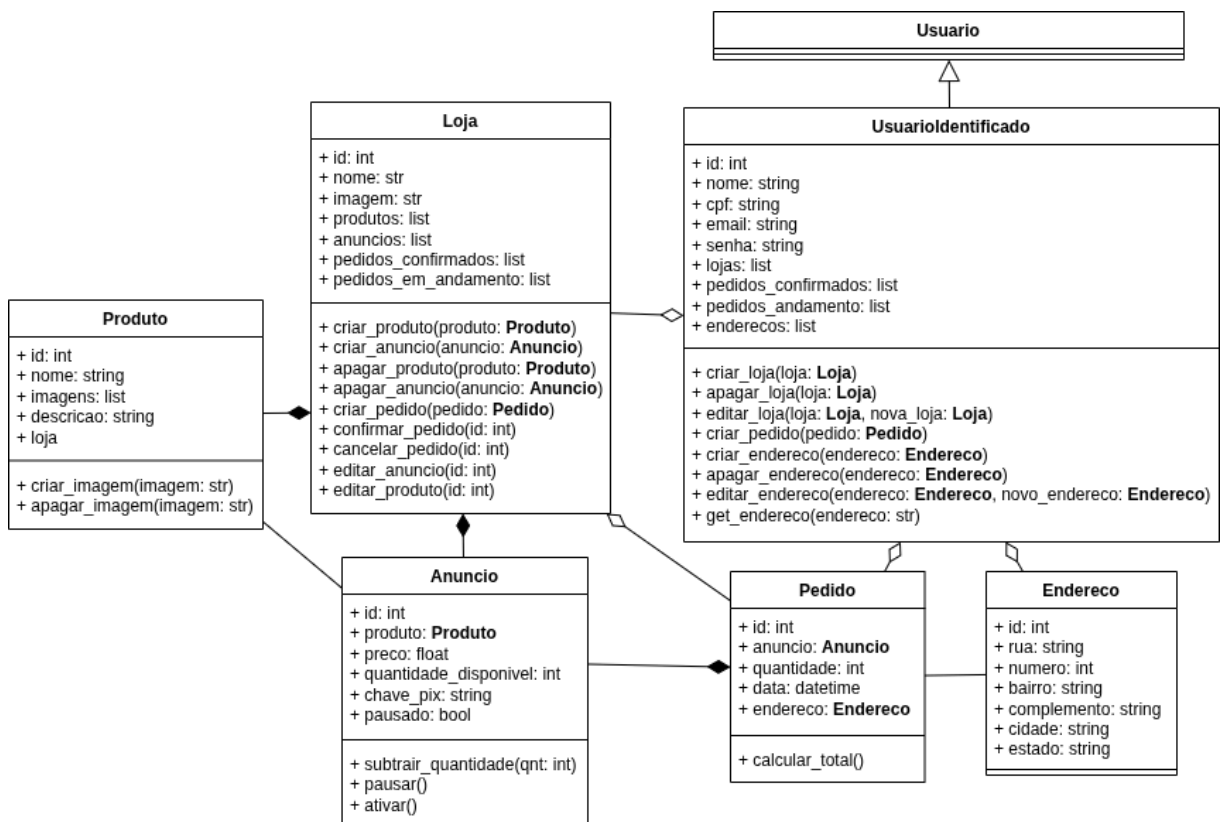


Figura: Diagrama de Classes

5. Vantagens :

O uso de middleware trouxe uma facilidade e simplicidade imensa em relação ao sockets. Detalhes como envio de imagem puderam ser tratados de forma muito mais simples, retornando o byte da imagem ou uma lista de bytes ao invés de ficar esperando as mensagens de forma separada. O código ficou muito mais limpo, pois uma requisição depende agora apenas da chamada de uma função. Além disso, o uso de uma abstração em nível mais alto revelou melhor que partes do código estavam acopladas demais, algo mais difícil de se perceber quando pensando em baixo nível.

6. Dificuldades :

As dificuldades encontradas foram em aprender como o middleware funciona e nos dissociar da ideia do socket, pois estávamos acostumados a pensar com uso de sockets e o middleware funciona de uma forma bem diferente, com facilidades e menos trabalho na unha. Além disso, as mudanças que foram necessárias no banco de dados fizeram com que o servidor de banco de dados tivesse uma maior mudança em relação aos outros processos, o que atrasou um pouco a implementação. Além disso, o processo de lidar com imagem em bytes sempre é um atraso grande na implementação.

7. Conclusão :

No final, obtivemos um sistema distribuído completo de Marketplace de poções mágicas, chamado Caldeirão, utilizando Python e o middleware Pyro5. Além disso, obtivemos muitos aprendizados e conhecimentos novos, que levaremos ao longo de toda nossa carreira como Cientistas da Computação.

8. Referências :

[1] PYRO5. *Pyro5 Documentation*. Disponível em: <https://pyro5.readthedocs.io/>. Acesso em: 14 jun. 2025.